

Assignment 3

5. Problem name: Movie Tickets

```
#include <iostream>

#include <string>

#include <unordered_map>

#include <set>

using namespace std;

class MovieTicket {

    unordered_map<int, int> ticketsAvailable;

    set<pair<int, int>> bookings;
```

```
public:
```

```
    bool BOOK(int X, int Y) {  
  
        if (ticketsAvailable.find(Y) ==  
ticketsAvailable.end()) {  
  
            ticketsAvailable[Y] = 100; // Each  
movie starts with 100 slots [cite: 27]  
  
        }  
  
        if (bookings.count({X, Y}) ||  
ticketsAvailable[Y] <= 0) {  
  
            return false; // Return false if  
already booked or sold out [cite: 33]  
  
        }
```

```
    bookings.insert({X, Y});

    ticketsAvailable[Y]--;

    return true;

}
```

```
bool CANCEL(int X, int Y) {

    if (bookings.count({X, Y}) == 0) {

        return false;

    }

    bookings.erase({X, Y});

    ticketsAvailable[Y]++;

    return true;

}
```

```
}
```

```
bool IS_BOOKED(int X, int Y) {
```

```
    return bookings.count({X, Y}) > 0;
```

```
}
```

```
int AVAILABLE_TICKETS(int Y) {
```

```
    if (ticketsAvailable.find(Y) ==  
ticketsAvailable.end()) {
```

```
        return 100;
```

```
    }
```

```
    return ticketsAvailable[Y];
```

```
}
```

```
};
```

```
int main() {
```

```
    MovieTicket mt;
```

```
    int Q;
```

```
    if (!(cin >> Q)) return 0;
```

```
    while (Q--> {
```

```
        string type;
```

```
        cin >> type;
```

```
        if (type == "BOOK") {
```

```
            int x, y; cin >> x >> y;
```

```
        cout << (mt.BOOK(x, y) ? "true" :  
"false") << endl;  
  
    } else if (type == "CANCEL") {  
  
        int x, y; cin >> x >> y;  
  
        cout << (mt.CANCEL(x, y) ?  
"true" : "false") << endl;  
  
    } else if (type == "IS_BOOKED") {  
  
        int x, y; cin >> x >> y;  
  
        cout << (mt.IS_BOOKED(x, y) ?  
"true" : "false") << endl;  
  
    } else if (type ==  
"AVAILABLE_TICKETS") {  
  
        int y; cin >> y;
```

```

        cout << mt.AVAILABLE_TICKETS(y) <<

endl;

    }

}

return 0;

}

```

Output

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
✖ Executing task: C:/Windows/System32/cmd.exe /d /c .\build\Debug\outDebug.exe
✖ Terminal will be reused by tasks, press any key to close it.

• ✖ Executing task: C:/Windows/System32/cmd.exe /d /c g++ -Wall -Wextra -Wpedantic -Wshadow -Wformat=2 -Wcast-align -Wconversion -Wsign-conversion -Wnull-dereference -g3 -O0 -c c:\Users\Hp\OneDrive\Desktop\gunnu\ass3.cpp -o .\build\Debug\ass3.o && g++ -Wall -Wextra -Wpedantic -Wshadow -Wformat=2 -Wcast-align -Wconversion -Wsign-conversion -Wnull-dereference -g3 -O0 .\build\Debug\ass3.o -o .\build\Debug\outDebug.exe
✖ Terminal will be reused by tasks, press any key to close it.

• ✖ Executing task: C:/Windows/System32/cmd.exe /d /c .\build\Debug\outDebug.exe

6
BOOK 101 5
true
BOOK 102 5
true
IS_BOOKED 101 5
true
AVAILABLE_TICKETS 5
98
CANCEL 101 5
true
AVAILABLE_TICKETS 5
99
✖ Terminal will be reused by tasks, press any key to close it.

```

6. Problem Name: Bank Balance

```
#include <iostream>

#include <string>

#include <unordered_map>


using namespace std;


class Bank {

private:

    unordered_map<int, int> accounts;


public:

    // Create or add to account
```



```
bool CREATE(int X, int Y) {  
  
    if (accounts.find(X) ==  
accounts.end()) {  
  
        accounts[X] = Y;  
  
        return true;  
  
    } else {  
  
        accounts[X] += Y;  
  
        return false;  
  
    }  
  
}
```

```
bool DEBIT(int X, int Y) {
```

```
        if (accounts.find(X) == accounts.end()
|| accounts[X] < Y) {

            return false;

        }

        accounts[X] -= Y;

        return true;

    }
```

```
bool CREDIT(int X, int Y) {
```

```
        if (accounts.find(X) ==  
accounts.end()) {  
  
            return false;  
  
        }  
  
        accounts[X] += Y;  
  
        return true;  
  
    }
```

```
int BALANCE(int X) {  
  
    if (accounts.find(X) ==  
accounts.end()) {
```

```
        return -1;

    }

    return accounts[X];

}

};

int main() {

    Bank myBank;

    int Q;

    if (!(cin >> Q)) return 0;
```

```
while (Q--) {  
  
    string command;  
  
    cin >> command;  
  
    if (command == "CREATE") {  
  
        int X, Y;  
  
        cin >> X >> Y;  
  
        cout << (myBank.CREATE(X, Y) ?  
"true" : "false") << endl;  
  
    }  
  
    else if (command == "DEBIT") {  
  
        int X, Y;  
  
        cin >> X >> Y;
```

```
        cout << (myBank.DEBIT(X, Y) ?  
"true" : "false") << endl;  
  
    }  
  
    else if (command == "CREDIT") {  
  
        int X, Y;  
  
        cin >> X >> Y;  
  
        cout << (myBank.CREDIT(X, Y) ?  
"true" : "false") << endl;  
  
    }  
  
    else if (command == "BALANCE") {  
  
        int X;  
  
        cin >> X;
```

```
        cout << myBank.BALANCE(X) << endl;

    }

}

return 0;

}
```

Output

```
-wsign-conversion -wnull-dereference -g3 -O0 .\build\Debug\prob.o -o .\build\Debug\out
```

* Terminal will be reused by tasks, press any key to close it.

* Executing task: C:/Windows/System32/cmd.exe /d /c .\build\Debug\outDebug.exe

6
CREATE 101 500
true
CREATE 101 100
false
DEBIT 101 200
true
CREDIT 101 50
true
BALANCE 101
450
BALANCE 102